

Skills as Morphisms

A Categorical Framework for Compositional Skill Synthesis in Physical AI

[Full Name]

Independent Researcher, Côte d'Ivoire

[email]

Technical note formalizing the *Action-Workflow* architecture

July 13, 2026

Abstract

Action-Workflow proposes an architecture for Physical AI in which a large language model acts not as a direct action planner but as an inference engine over required *capabilities*, which are grounded into reusable reinforcement-learning skills, composed into workflows, and accumulated in a lifelong procedural memory. This note develops a categorical formalization of that proposal. We define a base category $\mathcal{S}$ whose morphisms are skills in the sense of the options framework of Sutton, Precup and Singh [11]; equip it with a symmetric, partial monoidal structure capturing parallel composition of independent capabilities; and construct a three-layer functorial semantics — task category, skill category, control category — in which the LLM’s reasoning stage is precisely a monoidal functor and reinforcement learning is precisely the construction of a section over a fibration-like forgetful map. Procedural memory is modeled as a filtered colimit of subcategories, and compositional safety is reduced to the monoidality of a forgetful functor from a category of verified skills. Throughout, we separate what follows formally from what remains an open technical commitment, in the spirit of a research programme rather than a finished theory.

1 Introduction

The Action-Workflow proposal repositions the large language model in a robotic system: rather than emitting an executable action sequence, the LLM infers *which capabilities* a natural-language goal requires. Those capabilities are grounded into reinforcement-learning skills, composed into an executable workflow, and — once validated — folded into a procedural memory that the system draws on for future tasks instead of re-deriving behavior from scratch. A central claim of the proposal is that such skills “can be represented as morphisms in category theory, enabling formal composition, verification, modularity, and safety guarantees.” As stated, this is a promissory note: it names the right mathematical neighborhood without specifying the category, the composition rule, or the guarantee. This note tries to cash it in.

Concretely, we make five constructions precise:

1. A base category $\mathcal{S}$ of *skills*, whose objects are symbolic conditions on the shared world-model state and whose morphisms are closed-loop, terminating control policies in the style of the options framework (§3).
2. A *monoidal* structure on $\mathcal{S}$ capturing the parallel composition of skills over independent subsystems or resources, together with an explicit statement of when that independence assumption is doing real work (§4).

3. A *functorial semantics* linking three layers of the Action-Workflow pipeline — natural-language goals, skills, and low-level control — in which the LLM’s capability-inference step is a monoidal functor (formalizing the phrase “LLM as compiler of behavioral abstractions”), and reinforcement learning is the construction of a section over a forgetful map from controllers to the skills they realize (§5).
4. A model of the *procedural memory* as a filtered colimit of an increasing chain of subcategories of $\mathcal{S}$ — an Ind-object in the sense of Grothendieck — which gives “continuously evolving skill library” a precise, standard meaning (§6).
5. A account of *compositional verification*: a category $\mathcal{V}$ of certified skills with a forgetful functor $U : \mathcal{V} \rightarrow \mathcal{S}$, where the informal promise of “safety guarantees” is exactly the statement that U is monoidal — and where we isolate the specific compatibility condition on specifications (Assumption 7.2) that this requires and does not get for free (§7).

None of this constitutes a finished theory. The purpose of a formalization at this stage is not to prove theorems but to expose commitments: once “skill” means Definition 3.1 and “composition” means the categorical operation of §3, several claims in the original proposal turn into propositions with hypotheses (stated below), and others turn into open problems (§8) that a category-theoretic vocabulary lets us state precisely rather than gesture at.

2 Related Formal Frameworks

The idea that a robot’s temporally-extended behaviors should be first-class, reusable objects is old: the *options* framework of Sutton, Precup and Singh [11] defines a temporally abstract action as a triple of an initiation set, a policy, and a termination condition, and shows how such options compose. We take this as the semantic content of a “skill” and recast it as a hom-set in a category (§3); the categorical reading adds nothing operationally but makes composition, identity, and — once a monoidal product is added — independence into first-class, checkable structure rather than properties one verifies case by case.

Two papers by Bakirtzis, Savvas, Topcu and coauthors are the closest existing work on categorical reinforcement learning: *Categorical semantics of compositional reinforcement learning* [4] and *Reduce, Reuse, Recycle: Categories for Compositional Reinforcement Learning* [5], the latter showing empirically that viewing Markov decision processes through a categorical lens supports task decomposition and reuse on robotic-arm benchmarks. Our base category $\mathcal{S}$ (§3) can be seen as a coarser, options-level counterpart to their MDP-level categorical structure. Independently, *RoboCat* [2] applies a category-theoretic framework to robot interoperability via goal-oriented programming; its emphasis is cross-platform composability of robot programs rather than lifelong reuse of learned skills, which is complementary to what we develop here.

The claim that an LLM “compiles” goals into skills is, read literally, a claim about denotational semantics: a functor from a syntactic category to a semantic one that preserves composition. This is the oldest business of categorical logic and programming-language semantics, and Moggi’s use of monads to give a uniform categorical account of computational effects [9] is the standard reference point; we use it only as an analogy in §5; we do not construct a category of natural-language expressions with the rigor that a programming-language semantics would demand.

The monoidal structure we place on $\mathcal{S}$ (§4) is best understood through the lens of *resource theories*: Coecke, Fritz and Spekkens [6] show that a wide class of “what can be converted into what, and at what rate” questions is governed by a symmetric monoidal category (or preorder) of processes acting on resources. Reading a skill’s precondition and postcondition as resource states before and after is exactly this pattern. Where two resources genuinely do not interact, this is unproblematic; where they share an actuator, workspace, or energy budget,

the tensor product’s joint functoriality can fail, and the appropriate relaxation is a *premonoidal* category in the sense of Power and Robinson [10], originally introduced to give a semantics to programs with side effects. We use this in §4 to flag, rather than paper over, the independence assumption.

Because skills are stochastic, a fully measure-theoretic account of $\mathcal{S}$ properly lives inside a *Markov category*: Giry’s original construction of a monad of probability measures [8], the compositional account of Markov processes of Baez, Fong and Pollard [3], and the more recent representable Markov categories of Fritz, Gonda, Perrone and Rischel [7] together supply the machinery for treating stochastic skills as morphisms in a category rather than as an informal add-on. We use this literature only as a remark (Remark 3.4); developing $\mathcal{S}$ as an explicit subcategory of a Markov category is left for future work.

Finally, for compositional verification (§7) the closest template is very recent work by Aberlé [1], which builds a framework in which implementations compose along wiring diagrams and their verifications compose *with* them, the whole compositionality being captured by a monoidal functor from a category of specifications to a category of interfaces. Our Assumption 7.2 and Proposition 7.3 are a coarse, robotics-flavored instance of exactly this pattern.

This note’s own point of departure — an LLM agent that distills its experience into a reusable, self-improving skill library rather than replanning from raw trajectories every time — is SkillRL [12], whose *SkillBank* mechanism is, to our knowledge, the most direct precedent for the “procedural memory” of §6. SkillRL’s evaluation is confined to text-based agents in simulated environments (ALFWorld, WebShop); the contribution of Action-Workflow, and of this note in particular, is to ask what the same idea demands once the skills in question must be grounded in continuous, embodied, multimodal control, and to give that demand a categorical shape.

3 A Base Category of Skills

3.1 State conditions

Fix a measurable world-model state space $(\mathcal{X}, \Sigma)$: the shared latent representation produced by fusing vision, audio, language, touch and proprioception. Rather than working with arbitrary measurable sets, fix a vocabulary $\mathcal{P}_0 \subseteq \Sigma$ of *conditions* — the symbolic predicates over $\mathcal{X}$ that the reasoning layer is entitled to talk about (“gripper empty”, “cup on table”, “door closed”). $\mathcal{P}_0$ is exactly the interface between the sub-symbolic world model and the symbolic/LLM reasoning layer of the Action-Workflow pipeline; we treat it as given rather than derive it.

Definition 3.1 (Skill). A *skill* $\pi : A \rightarrow B$ between conditions $A, B \in \mathcal{P}_0$ is a triple $\pi = (\mu_\pi, \beta_\pi, \varepsilon_\pi)$ where μ_π is a closed-loop control policy defined at least on A , β_π is a termination rule, and $\varepsilon_\pi \in [0, 1)$ is a failure tolerance, such that executing μ_π from any state in A terminates, almost surely in finite time, in a state in B with probability at least $1 - \varepsilon_\pi$. This is precisely an *option* in the sense of Sutton, Precup and Singh [11], indexed by its precondition and postcondition rather than left as a free-floating object.

Two skills are identified when they induce the same sub-probability transition kernel on A (i.e. the same distribution over termination state and time); $\mathcal{S}$ is the category with objects $\mathcal{P}_0$ and $\text{Hom}_{\mathcal{S}}(A, B)$ the set of skills $A \rightarrow B$ under this identification.

Definition 3.2 (Sequential composition). For $\pi_1 : A \rightarrow B$ and $\pi_2 : B \rightarrow C$, the composite $\pi_2 \circ \pi_1 : A \rightarrow C$ executes μ_{π_1} to termination and then μ_{π_2} (*call-and-return* composition). The identity $\text{id}_A : A \rightarrow A$ is the zero-time policy that terminates immediately, always in A .

Proposition 3.3. $\mathcal{S}$ is a category, and $\varepsilon_{\pi_2 \circ \pi_1} \leq \varepsilon_{\pi_1} + \varepsilon_{\pi_2}$.

Proof sketch. Associativity holds because “run μ_1 , then μ_2 , then μ_3 ” does not depend on how the sequence is bracketed; identity laws are immediate from id_A terminating instantly. The failure bound is a union bound over the event that π_1 fails to reach B or, having reached B , π_2 fails to reach C . $\square$

The failure-probability inequality in Proposition 3.3 is worth dwelling on: composition in $\mathcal{S}$ is free syntactically (any two skills with matching interface compose) but not free semantically (reliability degrades, generally monotonically, with the length of a composed workflow). Any safety story built on top of $\mathcal{S}$ — including §7 — has to carry this bookkeeping explicitly rather than assume composed skills are as reliable as their parts.

Remark 3.4 (Toward a measure-theoretic $\mathcal{S}$). Definition 3.1 is deliberately kept at the level of the classical options framework. A fully measure-theoretic treatment would instead take $\mathcal{S}$ to be (a subcategory of) the Kleisli category of a probability monad on $\mathcal{X}$ — the Giry monad [8] — restricted to those Kleisli morphisms satisfying the termination-in- B -with-probability- $\geq 1 - \varepsilon$ condition; this is exactly the setting of *Markov categories*, as developed compositionally by Baez, Fong and Pollard [3] and, with representability conditions suited to continuous state spaces, by Fritz, Gonda, Perrone and Rischel [7]. We do not carry out this embedding here; it is the natural next step for anyone wanting to reason about $\mathcal{S}$ with full measure-theoretic rigor.

4 Monoidal Structure: Independence and Parallel Composition

Sequential composition alone cannot express that two skills can be executed *together* — e.g. a mobile base navigating while an arm manipulates. We add this via a monoidal product, at the cost of an explicit independence assumption.

Definition 4.1 (Tensor of conditions). Say $A, A' \in \mathcal{P}_0$ are *independent* if they constrain disjoint coordinates of the world-model state (disjoint subsystems, actuators, or resources). For independent A, A' , let $A \otimes A' \in \mathcal{P}_0$ be the conjunction “ A holds on its coordinates and A' holds on its coordinates,” and let $I \in \mathcal{P}_0$ be the always-true condition.

Definition 4.2 (Parallel composition). For $\pi_1 : A_1 \rightarrow B_1$ and $\pi_2 : A_2 \rightarrow B_2$ with A_1, A_2 independent (and hence, we assume, B_1, B_2 independent), $\pi_1 \otimes \pi_2 : A_1 \otimes A_2 \rightarrow B_1 \otimes B_2$ executes μ_{π_1} and μ_{π_2} concurrently on their respective coordinates.

Proposition 4.3. *Restricted to pairs of mutually independent conditions, $(\mathcal{S}, \otimes, I)$ is a symmetric monoidal category.*

Proof sketch. Bifunctoriality of $\otimes$, the associator, unitors and symmetry braiding all reduce to the fact that independent coordinates evolve, by assumption, without interference, so concurrent execution factors as a product of the two marginal executions in either order or grouping. $\square$

Remark 4.4 (Skills as a resource theory). Proposition 4.3 places $\mathcal{S}$ inside the general pattern identified by Coecke, Fritz and Spekkens [6]: a symmetric monoidal category whose morphisms convert one resource state into another, with $\otimes$ combining independent resources. Reading precondition/postcondition pairs as resource states makes “which capability contexts can be reached from which others, and by composing which skills” a resource-convertibility question in their sense.

Remark 4.5 (When independence fails). Independence is a modeling convenience, not a physical law: two arms sharing a workspace, or two skills drawing on a common energy or attention budget, interfere, and $\otimes$ as defined is then only *partially* functorial — functorial in each argument separately but not necessarily jointly, exactly the situation that motivated Power

and Robinson to introduce *premonoidal* categories for modeling programs with shared, order-sensitive side effects [10]. A more careful treatment of Action-Workflow’s “parallel” pipeline stage should replace Proposition 4.3’s blanket independence hypothesis with an explicit interference relation on $\mathcal{P}_0$ and a premonoidal (rather than monoidal) product outside of it; we flag this rather than resolve it (§8).

5 Functorial Semantics: Goals, Skills, and Control

The Action-Workflow pipeline moves through three levels of description: a natural-language goal, a composition of skills, and the low-level control that actually realizes them. We introduce a category for each and place a functor between each pair.

5.1 Three categories

Let $\mathcal{T}$, the *task category*, be the free symmetric monoidal category generated by the atomic subgoal tokens the LLM’s decomposition vocabulary can produce, with $;$ and $\parallel$ as its sequential and parallel operations; a morphism $g \rightarrow g'$ is a decomposition witness that g' is an admissible refinement of g into (possibly composite) subgoals. We emphasize that $\mathcal{T}$ is a syntactic scaffold that the LLM’s outputs populate, not a formalization of natural language itself; how raw language is mapped onto tokens of $\mathcal{T}$ is outside the scope of this note.

Let $\mathcal{C}$, the *control category*, have as objects concrete (possibly continuous, possibly partially observed) robot-and-environment states, and as morphisms raw closed-loop trajectories: the level at which RL policies and robot controllers actually operate.

5.2 The LLM as a monoidal functor

Definition 5.1 (Capability-inference functor). The functor $R : \mathcal{T} \rightarrow \mathcal{S}$ sends an atomic subgoal token to the condition-pair it requires and produces, and a decomposition morphism $g \rightarrow g'$ to the corresponding skill composite in $\mathcal{S}$, satisfying $R(g_1; g_2) = R(g_2) \circ R(g_1)$ and $R(g_1 \parallel g_2) = R(g_1) \otimes R(g_2)$ whenever the right-hand sides are defined: R is a (partial) strong monoidal functor.

This is what it means, precisely, to say the LLM is a “compiler of behavioral abstractions” rather than a planner: R plays exactly the role a denotational-semantics functor plays for a programming-language compiler, translating source-level (task) structure into target-level (skill) structure while preserving how that structure composes — the categorical semantics tradition inaugurated by Moggi’s treatment of computational effects as monads [9] is the nearest analogue, though we do not develop $\mathcal{T}$ with anything like the same rigor. The requirement that R *preserve* composition is not decorative: it is precisely what rules out an LLM that decomposes a goal correctly in isolation but produces an inconsistent composite plan, since well-typedness of the composite in $\mathcal{S}$ (matching post- and pre-conditions) becomes a necessary condition that can be checked before anything executes.

5.3 Grounding as a section, RL as its construction

Many concrete controllers can realize one abstract skill. We model this with a forgetful map $p : \mathcal{C} \rightarrow \mathcal{S}$ sending a trajectory to the skill it realizes (its actual precondition, postcondition, and empirical failure rate). We do not claim p is a Grothendieck fibration in the full technical sense; the property we actually use is weaker and easier to justify: for every skill $\pi \in \mathcal{S}$, the *fiber* $p^{-1}(\pi) \subseteq \mathcal{C}$ of controllers realizing π is non-empty once π has been successfully synthesized, and reinforcement learning is the process of searching that fiber for a controller optimizing some auxiliary objective (reward, robustness, energy).

Definition 5.2 (Grounding). A *grounding* of R is a (partial) section $\Gamma : \mathcal{T} \rightarrow \mathcal{C}$ with $p \circ \Gamma = R$, i.e. a choice, for every goal the LLM can decompose, of a concrete controller realizing the skill composite R prescribes.

$$\begin{array}{ccc} & & \mathcal{C} \\ & \nearrow \Gamma & \downarrow p \\ \mathcal{T} & \xrightarrow{R} & \mathcal{S} \end{array}$$

Reinforcement learning constructs Γ over a given skill by optimizing within its fiber; imitation learning constructs an initial, typically partial, section Γ_0 defined only over the sub-region of $\mathcal{C}$ covered by demonstration data, which RL then extends and refines to a section over the full fiber — exactly the common practice of behavior-cloning initialization followed by RL fine-tuning, now with a name for what is being constructed at each stage. The dashed arrow records that Γ is neither unique nor canonical: it is *built*, not derived, which is the categorical way of saying that grounding is a learning problem rather than a lookup.

6 Procedural Memory as a Filtered Colimit

Definition 6.1 (Skill library). For an experience index t (episode count, wall-clock time, or similar), let $\mathcal{L}_t \subseteq \mathcal{S}$ be the full subcategory whose objects are conditions the system has encountered and whose morphisms are skills synthesized and validated by stage t : the *skill library*.

Definition 6.2 (Distillation). A distillation step is an inclusion-of-subcategories functor $\mathcal{L}_t \hookrightarrow \mathcal{L}_{t+1}$: newly synthesized, validated skills are added; under the simplifying assumption of monotonic memory, none already present are removed (a skill’s certificate may later be strengthened, but the skill itself persists).

Definition 6.3 (Lifelong library). The lifelong skill library is $\mathcal{L}_\infty := \text{colim}_t \mathcal{L}_t$, the filtered colimit of the chain $\mathcal{L}_0 \hookrightarrow \mathcal{L}_1 \hookrightarrow \mathcal{L}_2 \hookrightarrow \dots$ in **Cat** — equivalently, $\mathcal{L}_\infty$ is an *Ind-object* of $\mathcal{S}$ in the sense of Grothendieck.

Concretely, any *finite* diagram of objects and morphisms of $\mathcal{L}_\infty$ already lives inside some single finite-stage $\mathcal{L}_t$ — this is the defining property of a filtered colimit, and it is exactly what licenses treating “the library right now” as a legitimate finite approximation to an unboundedly growing ideal, rather than an ad hoc snapshot of something that isn’t really there yet.

Remark 6.4 (Retrieval vs. synthesis as a lifting problem). Given a goal g with required skill-shape $R(g) : A \rightarrow B$, *retrieval* succeeds exactly when $R(g)$ already factors through the inclusion $\mathcal{L}_t \hookrightarrow \mathcal{S}$ for the current t — a lifting problem with a yes/no answer, in principle decidable by a lookup over $\mathcal{L}_t$. *Synthesis* (via LLM-guided imitation and reinforcement learning, §5) is invoked exactly when no such factorization exists, and its output is precisely the new morphism that extends $\mathcal{L}_t \hookrightarrow \mathcal{L}_{t+1}$. In this sense “does the robot already know how to do this” and “factor this morphism through the library subcategory” are the same question.

7 Compositional Verification and Safety

This section makes precise the last, and strongest, informal claim in the original proposal: that the categorical structure buys *safety guarantees*.

Definition 7.1 (Specification). A *specification* on morphisms $A \rightarrow B$ is a predicate $\varphi_{A,B}$ on skills $A \rightarrow B$ — e.g. “succeeds with probability $\geq 1 - \delta$ ”, “never visits an unsafe region $\mathcal{U} \subset \mathcal{X}$ ”, “expected energy cost $\leq E$.” A skill $\pi : A \rightarrow B$ is φ -*verified* if it carries a certificate witnessing $\varphi_{A,B}(\pi)$.

Assumption 7.2 (Compositionality of the specification language). The chosen specification language admits:

- (i) *sequential compatibility*: a rule combining certificates for $\varphi_{A,B}(\pi_1)$ and $\varphi_{B,C}(\pi_2)$ into a certificate for $\varphi_{A,C}(\pi_2 \circ \pi_1)$;
- (ii) *parallel compatibility*: a rule combining certificates for $\varphi_{A_1,B_1}(\pi_1)$ and $\varphi_{A_2,B_2}(\pi_2)$ into a certificate for $\varphi_{A_1 \otimes A_2, B_1 \otimes B_2}(\pi_1 \otimes \pi_2)$, for independent A_1, A_2 .

Assumption 7.2 is doing all the real work in this section, and we state it as a hypothesis rather than a fact for a reason: it is generally false for an arbitrarily chosen φ (a conjunction of two per-step safety properties need not imply a useful property of the composite without additional structure — e.g. a shared risk budget must be split, not duplicated, across composed skills), and establishing it for a specific, robotics-relevant φ is exactly the kind of technical work that a category-theoretic formulation lets us name but does not do for us.

Proposition 7.3. *Under Assumption 7.2, φ -verified skills together with their certificates form a category $\mathcal{V}$ with a forgetful functor $U : \mathcal{V} \rightarrow \mathcal{S}$, and U is symmetric monoidal: the square*

$$\begin{array}{ccc} \mathcal{V} \times \mathcal{V} & \xrightarrow{\otimes_{\mathcal{V}}} & \mathcal{V} \\ U \times U \downarrow & & \downarrow U \\ \mathcal{S} \times \mathcal{S} & \xrightarrow{\otimes_{\mathcal{S}}} & \mathcal{S} \end{array}$$

commutes up to coherent isomorphism.

Proof sketch. Part (i) of Assumption 7.2 supplies composition of certificates, making $\mathcal{V}$ a category and U a functor by construction (forgetting the certificate, remembering the skill); part (ii) supplies the coherence isomorphism witnessing that $U(\pi_1) \otimes_{\mathcal{S}} U(\pi_2) \cong U(\pi_1 \otimes_{\mathcal{V}} \pi_2)$. $\square$

Proposition 7.3 is the precise content behind “verified skills compose into verified skills”: composing two independently-certified skills yields a certified composite *without re-verifying the whole workflow from scratch*, because the certificate itself is built compositionally, not searched for anew. This is structurally the same move made, in a program-verification setting, by Aberlé’s recent framework, where compositional verification is likewise reduced to a monoidal functor from specifications to interfaces [1]; we see our Assumption 7.2 and Proposition 7.3 as the coarse-grained, robotics-flavored specialization of that same abstract pattern, rather than as an independent result. What is specific to Action-Workflow — and left open here — is choosing a concrete φ (e.g. built from control-barrier-style forward-invariance conditions on $\mathcal{U}$) for which Assumption 7.2 can actually be proved rather than posited.

8 Discussion: What This Buys You, and What Remains Open

8.1 What follows formally, given the definitions above

- **Type-safe composition.** A workflow is well-typed in $\mathcal{S}$ only if each skill’s postcondition matches the next skill’s precondition; this is a syntactic check, performable before anything executes, that rules out an entire class of composition errors for free.
- **Explicit, non-hidden reliability degradation.** Proposition 3.3 forces the failure probability of a composed workflow to be tracked, rather than silently assumed away, as workflows grow longer.
- **A precise meaning for “continuously evolving memory.”** §6 turns a metaphor into a filtered colimit with a standard, well-understood universal property.

- **Compositional safety, conditional on Assumption 7.2.** Proposition 7.3 shows exactly what must be true of a specification language for “verify once, compose freely” to hold — and, by the same token, exactly what to check before trusting it.
- **Clean interfaces between LLM, RL and IL.** §5 separates “what composition of capabilities is required” (R , an LLM problem) from “what controller realizes a given capability” (Γ , an RL/IL problem) as two different categorical constructions (a functor and a section), rather than one monolithic model doing both jobs implicitly.

8.2 What remains open

Open problem 8.1 (Grounding $\mathcal{P}_0$ in the world model). This note takes the vocabulary of conditions $\mathcal{P}_0$ as given. In Action-Workflow it must instead be extracted from a continuous, learned, multimodal world-model representation — an open, and non-trivial, symbol-grounding problem that categorical structure alone does not solve.

Open problem 8.2 (Checkable rather than assumed independence). Proposition 4.3 assumes independence between conditions being tensored; Remark 4.5 names the premonoidal relaxation but does not carry it out. A working system needs an explicit, checkable interference relation on $\mathcal{P}_0$, not a standing assumption.

Open problem 8.3 (A concrete, provably compatible specification language). Proposition 7.3 is conditional on Assumption 7.2. Instantiating φ — plausibly via control-barrier-function-style invariants for safety and a splittable risk/energy budget for reliability — and proving, not positing, sequential and parallel compatibility is the central piece of unfinished business in this note.

Open problem 8.4 (From Ind-colimit to data structure). §6 specifies the right abstract shape of a lifelong skill library; it says nothing about how to index $\mathcal{L}_t$ for efficient retrieval at scale, which is an engineering question the category theory is deliberately silent on.

Open problem 8.5 (Beyond idealized options). Definition 3.1 inherits the idealizations of the options framework — discrete termination events, full observability of whether A or B hold. Extending $\mathcal{S}$ to genuinely continuous, partially observed, hybrid (discrete/continuous) control without losing the composition and verification results above is unresolved.

9 Conclusion

The sentence this note set out to make precise was one line in a longer architectural proposal: that reusable skills can be represented as morphisms in category theory, enabling formal composition, verification, modularity, and safety guarantees. Made precise, that sentence turns into a base category of options-style skills; a partial monoidal structure whose independence hypothesis is now visible rather than implicit; a three-functor semantics in which “LLM as compiler” and “RL as grounding” are different, well-typed categorical constructions; a filtered-colimit account of lifelong procedural memory; and a compositional-safety proposition whose truth is conditional on a stated, checkable assumption about the specification language rather than asserted outright. None of this replaces the empirical work of building Action-Workflow; what it offers is a shared vocabulary — objects, morphisms, functors, colimits — in which the architecture’s own claims about itself can be stated sharply enough to be attacked, extended, or shown false, which is the most a formalization can responsibly promise at this stage of the research.

References

- [1] C. B. Aberlé. Compositional program verification with polynomial functors in dependent type theory. *arXiv preprint arXiv:2604.01303*, 2026.
- [2] A. Aguineldo, J. Bunker, B. S. Pollard, A. Shukla, A. Canedo, G. Quiros, W. Regli. RoboCat: A category theoretic framework for robotic interoperability using goal-oriented programming. *IEEE Transactions on Automation Science and Engineering*, 2022.
- [3] J. C. Baez, B. Fong, B. S. Pollard. A compositional framework for Markov processes. *Journal of Mathematical Physics*, 2016.
- [4] G. Bakirtzis, M. Savvas, U. Topcu. Categorical semantics of compositional reinforcement learning. *arXiv preprint arXiv:2208.13687*, 2022.
- [5] G. Bakirtzis, M. Savvas, R. Zhao, S. Chinchali, U. Topcu. Reduce, reuse, recycle: Categories for compositional reinforcement learning. In *ECAI 2024. arXiv preprint arXiv:2408.13376*.
- [6] B. Coecke, T. Fritz, R. W. Spekkens. A mathematical theory of resources. *Information and Computation*, 250:59–86, 2016.
- [7] T. Fritz, T. Gonda, P. Perrone, E. F. Rischel. Representable Markov categories and comparison of statistical experiments in categorical probability. *Theoretical Computer Science*, 2023.
- [8] M. Giry. A categorical approach to probability theory. In *Categorical Aspects of Topology and Analysis*, Lecture Notes in Mathematics 915, Springer, 1982.
- [9] E. Moggi. Notions of computation and monads. *Information and Computation*, 93(1):55–92, 1991.
- [10] J. Power, E. Robinson. Premonoidal categories and notions of computation. *Mathematical Structures in Computer Science*, 7(5):453–468, 1997.
- [11] R. S. Sutton, D. Precup, S. Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2):181–211, 1999.
- [12] P. Xia, J. Chen, H. Wang, J. Liu, K. Zeng, Y. Wang, S. Han, Y. Zhou, X. Zhao, H. Chen, et al. SkillRL: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*, 2026.